

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: IOT VÀ ỨNG DỤNG

Giảng viên : Nguyễn Quốc Uy

Sinh viên : Dương Việt Hoàng

Lớp : D21CQCN12-B

Mã sinh viên : B21DCCN384

Số điện thoại : 0396418880

Tháng 10/2024

Mục lục

CHƯƠNG 1: GIỚI THIỆU	4
1.1 Đặt vấn đề	4
1.2 Mục tiêu và phạm vi	4
Chương 2 Giao diện,thiết kế tổng quan	6
2.1. Giao diện	6
2.2 Thiết kế tổng thể	7
Chương 3: Chi tiết	8
3.1. Các thiết bị phần cứng	8
3.2 MQTT Broker	12
3.3. Backend	15
3.4. Frontend	16
Chương 4. Code	19
4.1 Phần cứng	19
4.2 Backend	27
4.3 Frontend	28

Lời cảm ơn

Đầu tiên, em xin gửi lời cảm ơn sâu sắc đến Học viện Công nghệ Bưu chính Viễn thông và khoa CNTT1 đã đưa môn học IoT và ứng dụng vào trong chương trình giảng dạy. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến giảng viên bộ môn thầy Nguyễn Quốc Uy đã dạy dỗ, rèn luyện và truyền đạt những kiến thức quý báu cho em trong suốt thời gian học tập vừa qua.

Trong thời gian tham dự lớp học của thầy, em đã được tiếp thu thêm nhiều kiến thức bổ ích, học tập được tinh thần làm việc hiệu quả, nghiêm túc. Đây thực sự là những điều rất cần thiết cho quá trình học tập và công tác sau này của em. Thêm vào đó, nhờ sự dẫn dắt và chỉ bảo của thầy, em đã thực hiện được một đề tài bài tập lớn hoàn chỉnh cho môn học này, em rất biết ơn điều đó.

Em xin chân thành cảm ơn, chúc thầy luôn mạnh khỏe và tiếp tục đạt được nhiều thành công trong cuộc sống!

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề

Với sự phát triển không ngừng của công nghệ, Internet of Things (IoT) đã và đang mở ra nhiều tiềm năng mới trong việc kết nối và tự động hóa các thiết bị điện tử trong đời sống hàng ngày. IoT cho phép các thiết bị có thể giao tiếp và thu thập dữ liệu môi trường một cách chính xác và nhanh chóng, từ đó đưa ra các hành động điều khiển phù hợp, mang lại nhiều lợi ích về hiệu suất, tiết kiệm năng lượng và tăng cường sự tiện nghi.

Trong bối cảnh nhu cầu về tiết kiệm năng lượng và kiểm soát môi trường sống ngày càng tăng, việc tích hợp các hệ thống đo đạc cảm biến và điều khiển tự động trở thành một xu hướng tất yếu. Dự án này được phát triển nhằm xây dựng một hệ thống IoT có khả năng đo lường các thông số môi trường như ánh sáng, độ ẩm, nhiệt độ, và điều khiển các thiết bị điện tử như đèn LED, quạt và điều hòa không khí một cách tự động và hiệu quả.

Hệ thống sẽ giúp người dùng theo dõi và kiểm soát các yếu tố môi trường xung quanh trong thời gian thực, từ đó tối ưu hóa việc sử dụng năng lượng, nâng cao chất lượng không gian sống và làm việc. Dự án này cũng mở ra những cơ hội nghiên cứu và phát triển về ứng dụng IoT trong các lĩnh vực như nhà thông minh, quản lý năng lượng và hệ thống tự động hóa trong công nghiệp.

1.2 Mục tiêu và phạm vi

1.2.1 Mục tiêu

Dự án này hướng tới việc xây dựng một hệ thống IoT hoàn chỉnh với các mục tiêu cụ thể như sau:

Thu thập dữ liệu môi trường: Đo đạc chính xác các thông số về ánh sáng, độ ẩm và nhiệt độ từ các cảm biến trong thời gian thực.

Điều khiển thiết bị tự động: Xây dựng hệ thống điều khiển đèn LED, quạt và điều hòa không khí dựa trên các ngưỡng giá trị của cảm biến hoặc theo lệnh điều khiển từ xa của người dùng.

Giao diện quản lý trực quan: Thiết kế một giao diện trực quan trên nền tảng web hoặc ứng dụng di động, giúp người dùng dễ dàng theo dõi và điều khiển các thiết bị từ xa.

Tiết kiệm năng lượng và tối ưu hóa hoạt động: Tối ưu hóa hệ thống điều khiển để đảm bảo các thiết bị hoạt động hiệu quả, tiết kiệm năng lượng dựa trên các điều kiện thực tế về môi trường.

Tích hợp công nghệ truyền thông không dây: Sử dụng các giao thức IoT như Wi-Fi, MQTT hoặc WebSocket để truyền dữ liệu từ cảm biến đến máy chủ và từ máy chủ đến giao diện người dùng.

1.2.2 Phạm vi của dự án bao gồm:

Các loại cảm biến: Hệ thống sẽ sử dụng cảm biến đo độ sáng, độ ẩm, và nhiệt độ để thu thập thông tin về môi trường.

Các thiết bị điều khiển: Hệ thống sẽ điều khiển đèn LED, quạt và điều hòa thông qua các công tắc điện tử được kết nối với hệ thống IoT.

Phạm vi không gian hoạt động: Hệ thống được triển khai trong một không gian nội thất như phòng ở hoặc văn phòng có diện tích vừa và nhỏ.

Truyền tải dữ liệu: Dữ liệu sẽ được truyền trong thời gian thực từ cảm biến đến server và từ server đến người dùng thông qua mạng không dây.

Giới hạn kỹ thuật: Dự án tập trung vào các kỹ thuật IoT cơ bản, không đi sâu vào các mô hình trí tuệ nhân tạo (AI) hoặc máy học (Machine Learning) trong điều khiển tự động.

Chương 2 Giao diện, thiết kế tổng quan

2.1. Giao diện

Giao diện, chúng ta sẽ mô tả chi tiết các màn hình chính của hệ thống IoT, bao gồm các chức năng và thiết kế của từng màn hình:

Màn hình Dashboard là trung tâm của giao diện, nơi người dùng có thể theo dõi và quản lý các chỉ số môi trường trong thời gian thực. Trên màn hình này, các thông số như nhiệt độ, độ ẩm và ánh sáng được hiển thị rõ ràng, cho phép người dùng theo dõi sự thay đổi của chúng theo thời gian. Màn hình còn bao gồm các biểu đồ thể hiện sự biến động của các thông số này, giúp người dùng dễ dàng nắm bắt xu hướng và nhận biết các biến đổi quan trọng. Bên cạnh đó, màn hình Dashboard còn cung cấp các nút điều khiển để bật/tắt các thiết bị điện như quạt, điều hòa và đèn, cho phép người dùng quản lý các thiết bị này một cách nhanh chóng và thuận tiện.

Màn hình Data Sensor cung cấp cái nhìn tổng quan về tất cả các thông số đo lường từ trước đến nay. Đây là nơi người dùng có thể xem và quản lý dữ liệu lịch sử của các cảm biến.

Màn hình Data Sensor bao gồm các chức năng phân trang để dễ dàng điều hướng qua nhiều dữ liệu, cùng với các công cụ tìm kiếm và sắp xếp để người dùng có thể nhanh chóng tìm thấy thông tin cần thiết. Các tùy chọn sắp xếp có thể được cấu hình để hiển thị dữ liệu theo các tiêu chí khác nhau như thời gian, loại cảm biến hoặc giá trị đo lường.

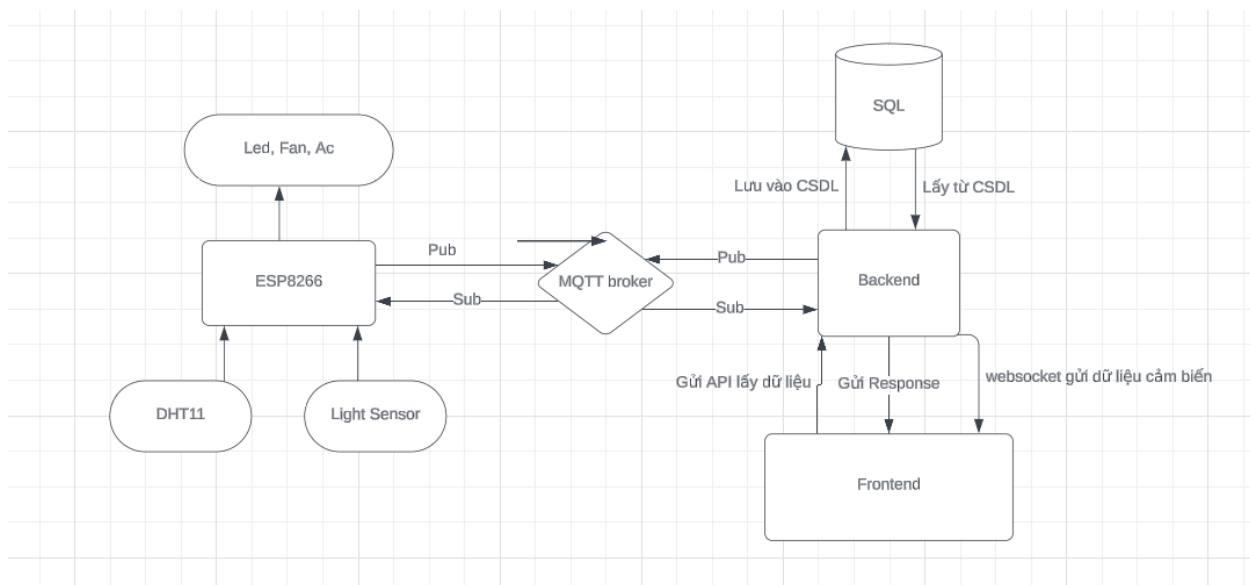
Màn hình Action History tập trung vào việc lưu trữ và quản lý lịch sử các hành động liên

quan đến việc bật/tắt các thiết bị điện. Màn hình này cho phép người dùng xem lại tất cả các hành động đã được thực hiện, với các tùy chọn phân trang và tìm kiếm theo khoảng thời gian để dễ dàng theo dõi các sự kiện cụ thể. Bên cạnh đó, người dùng cũng có thể lọc dữ liệu theo từng thiết bị, giúp dễ dàng xác định các thay đổi và hoạt động của từng thiết bị trong hệ thống.

Màn hình Profile chứa thông tin chi tiết của sinh viên sử dụng hệ thống. Màn hình này

cung cấp một cái nhìn tổng quan về thông tin cá nhân, bao gồm tên, ảnh đại diện, và các thông tin liên quan khác như github và báo cáo. Đây là nơi người dùng có thể cập nhật và quản lý thông tin cá nhân của mình, đảm bảo rằng các dữ liệu liên quan đến người sử dụng được giữ cập nhật và chính xác.

2.2 Thiết kế tổng thể

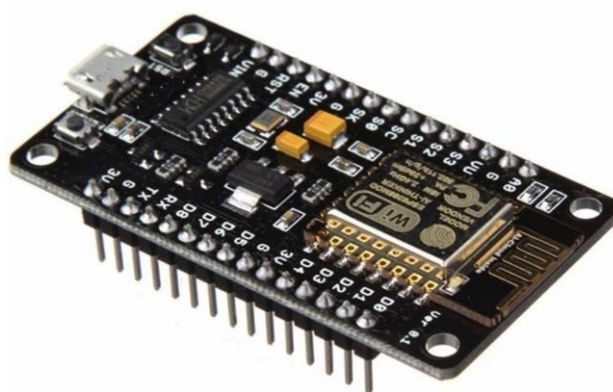


Chương 3: Chi tiết

3.1. Các thiết bị phần cứng

3.1.1 ESP8266

ESP8266 là một vi điều khiển tích hợp Wi-Fi có giá thành thấp, rất phổ biến trong các dự án IoT nhờ khả năng kết nối mạng không dây và xử lý dữ liệu mạnh mẽ. Được phát triển bởi Espressif Systems, ESP8266 được đánh giá cao nhờ vào kích thước nhỏ gọn, dễ lập trình và khả năng giao tiếp linh hoạt với các cảm biến và thiết bị ngoại vi.



Hình ảnh về ESP8266

Đặc điểm chính:

- Kết nối Wi-Fi tích hợp: ESP8266 được tích hợp sẵn module Wi-Fi, giúp nó dễ dàng kết nối với các mạng không dây mà không cần thêm bất kỳ phần cứng nào khác.
- Khả năng lập trình dễ dàng: Vi điều khiển này có thể được lập trình bằng nhiều ngôn ngữ khác nhau như Lua, Python, và phổ biến nhất là Arduino IDE, tạo điều kiện thuận lợi cho các nhà phát triển từ nhiều cấp độ kỹ năng khác nhau.

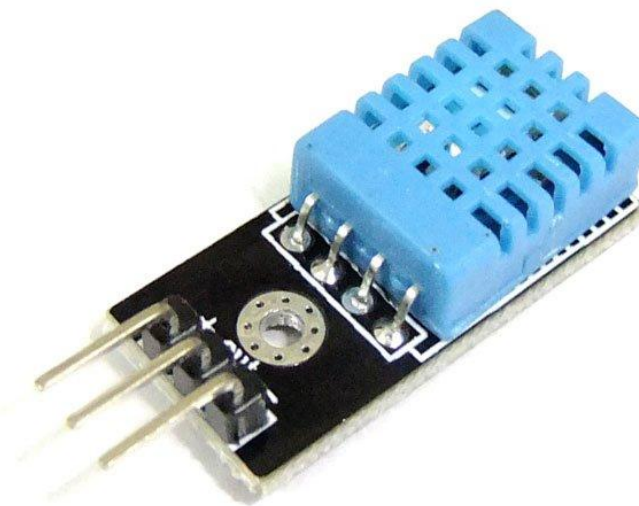
- Tích hợp CPU và bộ nhớ: ESP8266 có một CPU 32-bit, tốc độ lên đến 80 MHz và bộ nhớ Flash 1 MB, đủ để xử lý các tác vụ trong các ứng dụng IoT.
- Các giao thức truyền thông: Hỗ trợ các giao thức như I2C, SPI, UART, giúp dễ dàng kết nối và trao đổi dữ liệu với các cảm biến và module ngoại vi khác.
- Tiết kiệm năng lượng: ESP8266 có chế độ ngủ (deep sleep) để tiết kiệm năng lượng, rất phù hợp cho các ứng dụng IoT yêu cầu hoạt động lâu dài mà vẫn duy trì kết nối mạng.
- Giá thành rẻ: So với các vi điều khiển khác có khả năng kết nối Wi-Fi, ESP8266 có giá thành thấp, là lựa chọn hợp lý cho các dự án quy mô nhỏ hoặc cá nhân.

Ứng dụng trong dự án:

- Trong dự án này, ESP8266 đóng vai trò quan trọng trong việc thu thập dữ liệu từ các cảm biến ánh sáng, độ ẩm, nhiệt độ và truyền thông tin đó đến server qua Wi-Fi. Ngoài ra, nó còn điều khiển các thiết bị điện tử như đèn LED, quạt, và điều hòa từ xa thông qua các lệnh được gửi từ giao diện người dùng. Với khả năng kết nối mạnh mẽ và tiết kiệm chi phí, ESP8266 là giải pháp lý tưởng cho các hệ thống IoT quy mô nhỏ như dự án này.

3.1.2 DHT11

DHT11 là một loại cảm biến giá rẻ và phổ biến dùng để đo nhiệt độ và độ ẩm trong không khí. Với thiết kế đơn giản và dễ sử dụng, DHT11 thường được sử dụng trong các dự án IoT, tự động hóa, và các ứng dụng liên quan đến việc giám sát môi trường.



Hình ảnh về DHT11

Đặc điểm chính:

- Đo nhiệt độ và độ ẩm: DHT11 có khả năng đo nhiệt độ trong khoảng từ 0°C đến 50°C với độ chính xác $\pm 2^\circ\text{C}$ và độ ẩm trong khoảng từ 20% đến 90% với độ chính xác $\pm 5\%$.
- Giao tiếp đơn giản: DHT11 sử dụng giao thức giao tiếp một dây (single-wire) để truyền dữ liệu, giúp kết nối dễ dàng với các vi điều khiển như ESP8266.
- Tần suất lấy mẫu hợp lý: DHT11 có thời gian lấy mẫu khoảng 1 giây, đủ cho các ứng dụng theo dõi nhiệt độ và độ ẩm theo thời gian thực mà không tiêu tốn quá nhiều tài nguyên hệ thống.
- Độ bền cao: Cảm biến DHT11 có tuổi thọ cao, hoạt động ổn định trong nhiều điều kiện môi trường khác nhau, rất phù hợp cho các ứng dụng giám sát lâu dài.
- Kích thước nhỏ gọn: Với kích thước nhỏ và tiêu thụ điện năng thấp, DHT11 dễ dàng tích hợp vào các hệ thống IoT có không gian hạn chế hoặc yêu cầu tiết kiệm năng lượng.

Ứng dụng trong dự án:

- Trong dự án này, DHT11 được sử dụng để đo các thông số về nhiệt độ và độ ẩm trong môi trường. Các dữ liệu này sẽ được gửi đến ESP8266 để xử lý và hiển thị lên giao diện người dùng. DHT11 giúp người dùng có thể theo dõi tình trạng môi trường trong thời gian thực, từ đó có các biện pháp điều chỉnh thích hợp để tối ưu hóa điều kiện sống, như bật tắt quạt hoặc điều hòa một cách tự động dựa trên nhiệt độ và độ ẩm đo được.

3.1.3 Cảm biến ánh sáng

Cảm biến ánh sáng (Light Sensor) là thiết bị dùng để đo cường độ ánh sáng trong môi trường. Cảm biến này thường được sử dụng trong các ứng dụng IoT để điều chỉnh mức độ chiếu sáng hoặc kích hoạt các hành động tự động dựa trên ánh sáng môi trường. Một trong những loại cảm biến ánh sáng phổ biến cho các dự án IoT là cảm biến quang trở (LDR - Light Dependent Resistor).



Cảm biến ánh sáng

Đặc điểm chính:

- Nguyên lý hoạt động: Cảm biến ánh sáng hoạt động dựa trên nguyên lý thay đổi điện trở theo cường độ ánh sáng. Khi ánh sáng chiếu vào, điện trở của cảm biến sẽ giảm, và khi ánh sáng yếu đi, điện trở tăng lên. Tín hiệu điện này sẽ được đọc và chuyển đổi thành giá trị ánh sáng để xử lý.

- Dải đo cường độ ánh sáng: Cảm biến có thể đo cường độ ánh sáng từ mức rất thấp (ánh sáng yếu) đến rất cao (ánh sáng mạnh) tùy thuộc vào loại cảm biến được sử dụng. Đối với LDR, nó có thể đo ánh sáng tự nhiên trong các điều kiện khác nhau như ban ngày, ban đêm hoặc trong môi trường trong nhà.
- Tính tương thích cao: Cảm biến ánh sáng dễ dàng kết nối với các vi điều khiển như ESP8266 thông qua các chân analog hoặc digital, giúp thu thập dữ liệu ánh sáng một cách liên tục và nhanh chóng.
- Ứng dụng đa dạng: Cảm biến ánh sáng thường được sử dụng trong các hệ thống tự động điều chỉnh độ sáng đèn, hệ thống chiếu sáng thông minh, và các thiết bị tiết kiệm năng lượng. Nó cũng đóng vai trò quan trọng trong các hệ thống cảnh báo, điều chỉnh dựa trên điều kiện ánh sáng xung quanh.

Ứng dụng trong dự án:

- Trong dự án này, cảm biến ánh sáng được sử dụng để đo cường độ ánh sáng trong không gian làm việc hoặc sinh hoạt. Dữ liệu ánh sáng này sẽ được gửi về ESP8266 để xử lý và điều khiển đèn LED tự động, bật/tắt theo mức độ ánh sáng môi trường. Khi ánh sáng tự nhiên yếu, hệ thống sẽ tự động bật đèn để đảm bảo đủ ánh sáng cho không gian, và ngược lại, khi ánh sáng đủ mạnh, hệ thống sẽ tắt đèn để tiết kiệm năng lượng. Cảm biến ánh sáng giúp tối ưu hóa việc sử dụng năng lượng chiếu sáng và nâng cao sự tiện nghi cho người sử dụng.

3.2 MQTT Broker

3.2.1 Giới thiệu

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, được thiết kế cho các ứng dụng IoT có yêu cầu truyền dữ liệu hiệu quả và đáng tin cậy giữa các thiết bị có băng thông thấp và tài nguyên giới hạn. MQTT Broker đóng vai trò quan trọng trong hệ thống MQTT, làm nhiệm vụ trung gian quản lý việc trao đổi dữ liệu giữa các thiết bị (client).

Đặc điểm chính của MQTT Broker:

- Hoạt động dựa trên mô hình "Publish/Subscribe": Trong hệ thống MQTT, các thiết bị đóng vai trò là client có thể đăng ký (subscribe) hoặc xuất bản (publish) dữ liệu vào các "chủ đề" (topics) thông qua MQTT Broker. Broker sẽ nhận dữ liệu từ các thiết bị đăng xuất bản và chuyển nó đến các thiết bị đã đăng ký vào cùng chủ đề đó.
- Tối ưu hóa băng thông: MQTT được thiết kế để hoạt động trong các môi trường có băng thông thấp hoặc không ổn định, nhờ kích thước gói dữ liệu nhỏ và khả năng duy trì kết nối ổn định ngay cả khi có sự gián đoạn mạng.
- Độ tin cậy cao: MQTT Broker đảm bảo việc phân phối tin nhắn theo các mức chất lượng dịch vụ (QoS - Quality of Service), từ mức giao hàng không bảo đảm đến mức giao hàng chính xác một lần (exactly once), giúp dữ liệu luôn được truyền đạt đúng và đủ.
- Khả năng mở rộng: Các MQTT Broker như Mosquitto hoặc EMQX có thể hỗ trợ hàng ngàn client kết nối cùng lúc, cho phép mở rộng hệ thống IoT từ quy mô nhỏ đến lớn một cách dễ dàng.
- Bảo mật: MQTT hỗ trợ nhiều cơ chế bảo mật như mã hóa SSL/TLS và cơ chế xác thực client, đảm bảo tính an toàn cho việc truyền dữ liệu trong hệ thống IoT.

Ứng dụng trong dự án:

- Trong dự án này, MQTT Broker đóng vai trò là trung gian để quản lý việc truyền dữ liệu giữa các cảm biến và các thiết bị điều khiển như đèn LED, quạt và điều hòa không khí. Cụ thể:
- Các cảm biến đo ánh sáng, độ ẩm và nhiệt độ sẽ gửi dữ liệu lên MQTT Broker theo các chủ đề (topics) đã định nghĩa trước.
- Broker sẽ chuyển tiếp các thông tin này đến các client đã đăng ký (ví dụ: ứng dụng điều khiển trên web hoặc di động) để hiển thị dữ liệu môi trường.
- Đồng thời, các lệnh điều khiển từ giao diện người dùng cũng sẽ được gửi qua MQTT Broker đến các thiết bị như đèn, quạt và điều hòa để thực hiện hành động tương ứng (bật/tắt).

- Với MQTT Broker, hệ thống có thể hoạt động hiệu quả trong thời gian thực, đảm bảo dữ liệu được truyền nhanh chóng, đồng thời tiết kiệm băng thông và năng lượng, đặc biệt là trong các ứng dụng IoT có quy mô lớn hoặc nhiều thiết bị.

3.2.2 Cấu hình MQTT

```
listener 1883  
allow_anonymous false  
password_file E:\mosquitto\passwd.txt
```

Trong cấu hình của broker MQTT, dòng lệnh sau được sử dụng để thiết lập các thông số quan trọng:

- `listener 1883`: Dòng lệnh này cấu hình broker MQTT để lắng nghe cổng 1883. Cổng 1883 là mặc định dành cho giao thức MQTT không mã hóa. Điều này giới hạn việc truy cập broker chỉ từ những thiết bị trên cùng mạng nội bộ có địa chỉ IP trong phạm vi phù hợp, giúp tăng cường bảo mật và kiểm soát truy cập.
- `password_file passwd`: Cấu hình này chỉ ra rằng broker sẽ sử dụng một tập tin mật khẩu (`passwd`) để quản lý xác thực người dùng. Mỗi thiết bị hoặc ứng dụng khi kết nối tới broker phải cung cấp tên người dùng và mật khẩu đúng với thông tin lưu trong tập tin này. Điều này đảm bảo rằng chỉ những người dùng hợp lệ mới có thể kết nối và gửi/nhận thông điệp qua broker.
- `allow_anonymous false`: Tùy chọn này được đặt thành `false`, nghĩa là broker không cho phép kết nối ẩn danh. Tất cả các kết nối phải thông qua quá trình xác thực bằng tên người dùng và mật khẩu. Điều này là cần thiết để đảm bảo rằng chỉ có những thiết bị và người dùng được cấp quyền mới có thể truy cập vào hệ thống, giúp tăng cường bảo mật và ngăn chặn các truy cập trái phép.
- Với file `passwd` đã cấu hình xong `username` và `password` (mật khẩu đã được mã hóa).

3.3. Backend

3.3.1. Giới thiệu

Trong dự án này, backend được xây dựng bằng ExpressJS, một thư viện phổ biến của Node.js dùng để phát triển các ứng dụng web. ExpressJS cung cấp một bộ công cụ mạnh mẽ nhưng đơn giản để phát triển các API và xử lý yêu cầu HTTP. Trong hệ thống IoT, backend đóng vai trò quan trọng trong việc quản lý dữ liệu và điều khiển các thiết bị thông qua giao thức MQTT và cơ sở dữ liệu MySQL.

3.3.2. Chức năng của Backend

Backend thực hiện các nhiệm vụ chính sau:

Trao đổi dữ liệu với hardware thông qua broker MQTT:

- Đóng vai trò client: Backend hoạt động như một client trong hệ thống MQTT, liên kết với broker MQTT để gửi và nhận các thông điệp. Backend nhận các thông số từ các thiết bị phân cứng như cảm biến nhiệt độ, độ ẩm, và ánh sáng, hoặc gửi yêu cầu điều khiển từ frontend (chẳng hạn như bật/tắt quạt, đèn, điều hòa) đến phân cứng.

- Xử lý dữ liệu: Các dữ liệu từ hardware sau khi được backend nhận sẽ được xử lý và có thể được lưu trữ vào cơ sở dữ liệu hoặc phản hồi lại cho frontend.

Giao tiếp với cơ sở dữ liệu (SQL):

- Lưu trữ dữ liệu: Backend giao tiếp với SQL để lưu trữ các thông số từ các cảm biến và trạng thái thiết bị. Điều này giúp duy trì một bản ghi liên tục của các giá trị môi trường cũng như lịch sử điều khiển thiết bị. Đồng thời cũng lưu dữ liệu lịch sử bật/tắt các thiết bị.

- Truy xuất dữ liệu: Backend cũng cho phép truy xuất dữ liệu từ MongoDB theo yêu cầu từ frontend, chẳng hạn như lấy các thông số đã lưu trong một khoảng thời gian cụ thể để phân tích hoặc hiển thị biểu đồ.

Nhận request từ frontend và trả lại response tương ứng:

- Xử lý request HTTP: Backend nhận các request từ frontend, như yêu cầu lấy dữ liệu cảm biến, yêu cầu điều khiển thiết bị, hoặc truy vấn lịch sử dữ liệu.

- Trả lại response: Dựa trên request nhận được, backend xử lý và gửi lại response tương ứng cho frontend, đảm bảo rằng người dùng có thể theo dõi các thông số môi trường hoặc điều khiển các thiết bị một cách chính xác và kịp thời.

Backend trong hệ thống IoT của em được xây dựng với ExpressJS và đóng vai trò then chốt trong việc quản lý và điều phối dữ liệu giữa phần cứng, cơ sở dữ liệu, và giao diện người dùng. Bằng cách sử dụng MQTT để giao tiếp với phần cứng và MongoDB để lưu trữ dữ liệu, backend đảm bảo rằng hệ thống hoạt động hiệu quả và ổn định, đáp ứng tốt các yêu cầu của ứng dụng IoT.

Gửi dữ liệu cảm biến qua Websocket đến frontend.

3.4. Frontend

3.4.1. Giới thiệu

Frontend của hệ thống IoT này được xây dựng bằng ReactJS, một thư viện JavaScript nổi tiếng dùng để phát triển giao diện người dùng (UI). ReactJS cung cấp khả năng tạo ra các ứng dụng web động và phản hồi nhanh, lý tưởng cho việc hiển thị dữ liệu thời gian thực từ các thiết bị IoT. Frontend được thiết kế để người dùng có thể dễ dàng theo dõi các thông số môi trường, quản lý thiết bị và xem lại lịch sử hành động thông qua các trang giao diện khác nhau.

Các trang giao diện chính

Frontend bao gồm bốn trang giao diện chính, mỗi trang thực hiện các chức năng cụ thể:

Dashboard:

- Chức năng: Trang Dashboard cung cấp một cái nhìn tổng quan về toàn bộ hệ thống, hiển thị các thông số chính như nhiệt độ, độ ẩm, ánh sáng theo thời gian thực. Dashboard cũng có thể hiển thị trạng thái hiện tại của các thiết bị như quạt, đèn, điều hòa. Đồng thời là biểu đồ thay đổi các thông số.

- Tương tác: Người dùng có thể thực hiện các thao tác bật/tắt thiết bị trực tiếp từ Dashboard và nhận phản hồi ngay lập tức.

Data Sensor:

- Chức năng: Trang Data Sensor hiển thị chi tiết các dữ liệu từ các cảm biến như nhiệt độ, độ ẩm, và ánh sáng. Các dữ liệu này có thể được hiển thị dưới dạng bảng, cho phép người dùng theo dõi xu hướng và biến động của các thông số môi trường theo thời gian.

- Tương tác: Người dùng có thể chọn khoảng thời gian để xem dữ liệu lịch sử hoặc tìm kiếm theo các thông số.

History Action:

- Chức năng: Trang History Action ghi lại và hiển thị lịch sử các hành động điều khiển thiết bị, như việc bật/tắt quạt, đèn, hoặc điều hòa. Người dùng có thể theo dõi những thay đổi đã được thực hiện và thời gian thực hiện.

- Tương tác: Lịch sử hành động được phân loại theo thời gian, cho phép người dùng dễ dàng tìm kiếm và xem lại các hành động đã thực hiện.

Profile:

- Chức năng: Trang Profile cung cấp thông tin cá nhân của người dùng, bao gồm thông tin về tài khoản, các tùy chọn cài đặt và khả năng quản lý thông tin cá nhân.

3.4.2. Chức năng tổng quan của Frontend

Frontend đảm nhận vai trò giao tiếp với backend để xử lý và hiển thị dữ liệu cho người dùng. Các chức năng chính bao gồm:

Gửi request cho backend:

- Frontend gửi các request HTTP tới backend để lấy dữ liệu thời gian thực từ các cảm biến, dữ liệu lịch sử (history action), và thông tin từ trang Data Sensor.

- Ngoài ra, frontend cũng gửi các yêu cầu bật/tắt thiết bị từ giao diện người dùng đến backend để điều khiển phần cứng.

Nhận và hiển thị response:

- Sau khi nhận được phản hồi từ backend, frontend cập nhật giao diện người dùng với các thông tin mới nhất, như giá trị cảm biến hiện tại, trạng thái thiết bị, và thông tin lịch sử hành động.

- Dữ liệu được hiển thị một cách trực quan và dễ hiểu, giúp người dùng có thể theo dõi và điều khiển hệ thống một cách hiệu quả.

Chương 4. Code

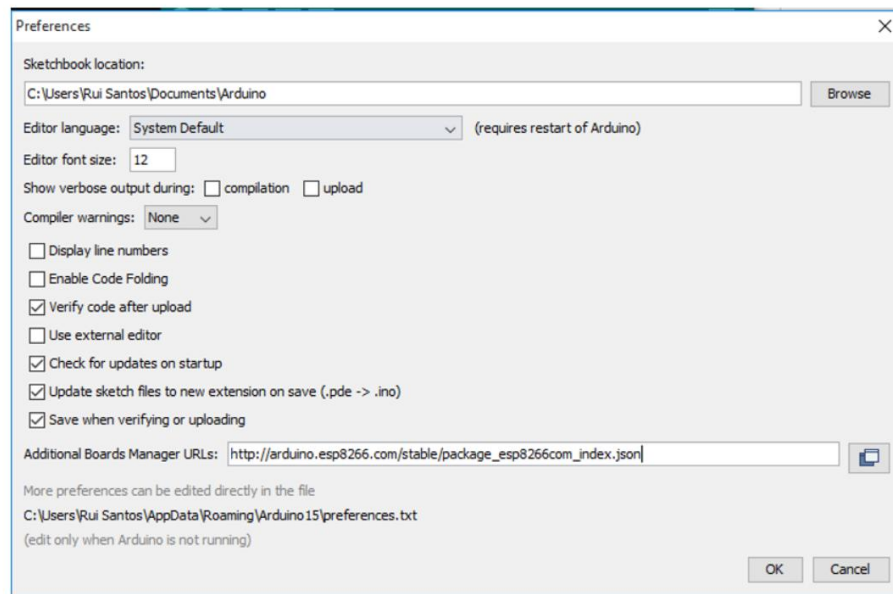
4.1 Phần cứng

Ở đây chúng ta dùng IDE Arduino để nạp code cho ESP8266. Để tiến hành cài đặt thư viện và chức năng nạp code cho ESP ta làm như sau:

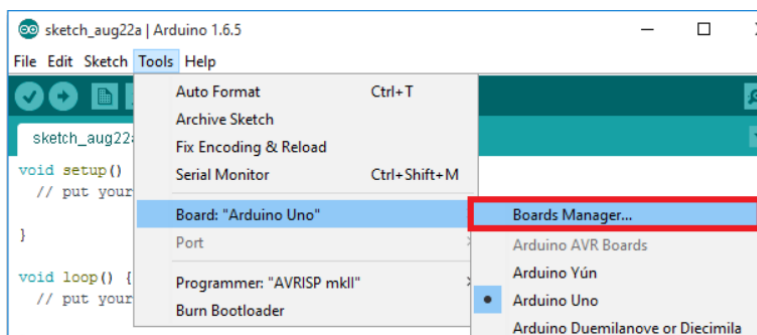
Vào File→ Preferences, vào textbox Additional Board Manager URLs thêm đường link sau vào:

http://arduino.esp8266.com/stable/package_esp8266com_index.json

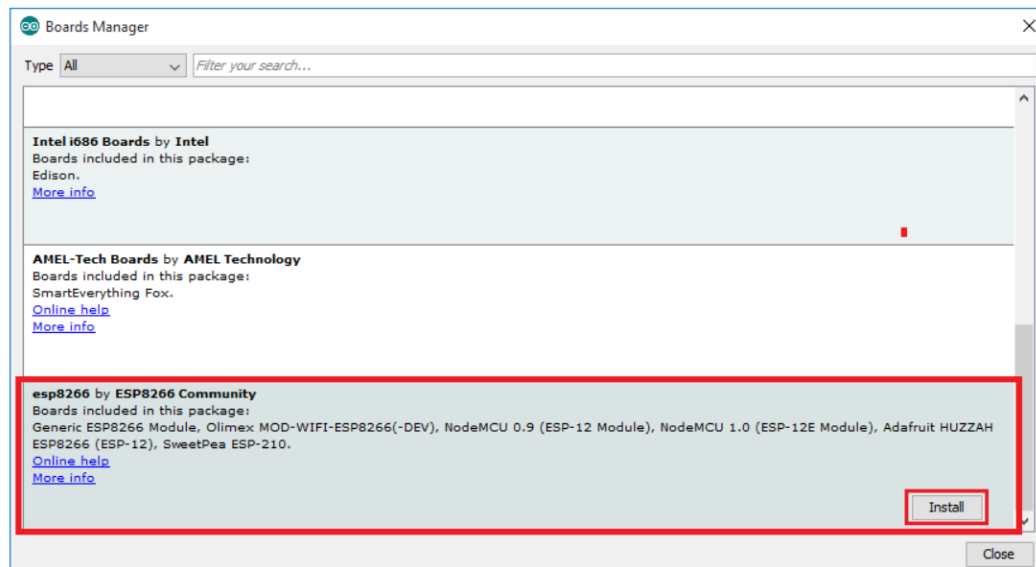
Click **OK** để chấp nhận.



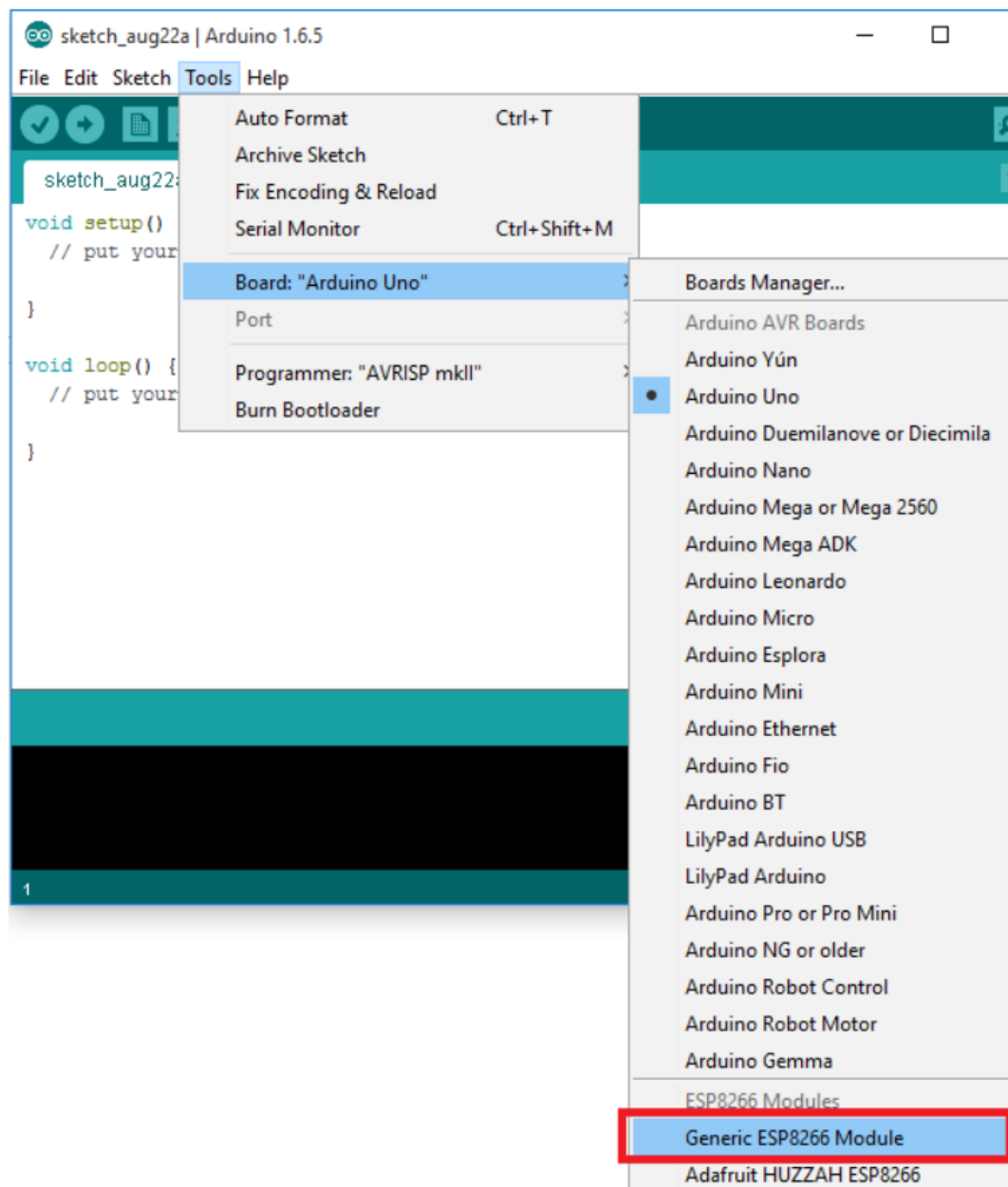
Tiếp theo vào Tool→Board→Boards Manager:



Đợi một lát để chương trình tìm kiếm. Ta kéo xuống và click vào ESP8266 by ESP8266 Community, click vào Install. Chờ phần mềm tự động download và cài đặt.



Chọn Board để lập trình cho ESP8266: Kết nối module USB-to-UART vào máy tính. Vào *Tool*→*Board*→*Generic ESP8266 Module*, chọn cổng COM tương ứng với module USB-to-UART tương ứng.



- Khai báo thư viện

```

1
2 #include <ESP8266WiFi.h>
3 #include <MQTTClient.h>
4 #include <ArduinoJson.h>
5 #include <DHT.h>
6
7 #define LED_PIN D2 // The ESP8266 pin D5 connected to resistor
8 #define FAN_PIN D3
9 #define AC_PIN D4
10 #define WARN_PIN D0
11 #define DHT_SENSOR_PIN D7 // The ESP8266 pin D7 connected to DHT11 sensor
12 #define DHT_SENSOR_TYPE DHT11
13 #define LIGHT_SENSOR_PIN A0
14
15 #define SUBSCRIBE_TOPIC "esp8266-001/receive"
16
17 DHT dht_sensor(DHT_SENSOR_PIN, DHT_SENSOR_TYPE);
18
19 const char WIFI_SSID[] = "DvH"; // CHANGE TO YOUR WIFI SSID
20 const char WIFI_PASSWORD[] = "12345678"; // CHANGE TO YOUR WIFI PASSWORD
21
22 const char MQTT_BROKER_ADDRESS[] = "172.20.10.4"; // CHANGE TO MQTT BROKER'S ADDRESS
23 const int MQTT_PORT = 1883;
24 const char MQTT_CLIENT_ID[] = "dvh"; // CHANGE IT AS YOU DESIRE
25 const char MQTT_USERNAME[] = "hoang"; // CHANGE IT IF REQUIRED, empty if not required
26 const char MQTT_PASSWORD[] = "b21dccn384"; // CHANGE IT IF REQUIRED, empty if not required
27
28 // The MQTT topics that ESP8266 should publish/subscribe
29 const char PUBLISH_TOPIC[] = "datasensor"; // CHANGE IT AS YOU DESIRE
30 const int PUBLISH_INTERVAL = 10000; // 5 seconds
31 unsigned long previousMillis = 0; // Thời gian lưu trạng thái trước đó
32 const long interval = 1000; // Khoảng thời gian nhấp nháy (1 giây)
33 bool ledState = LOW;
34 bool isWarningOn = false; // Cờ để kiểm tra xem đèn cảnh báo có đang bật hay không
35 WiFiClient network;
36 MQTTClient mqtt = MQTTClient(256);
37
38 unsigned long lastPublishTime = 0;

```

1. void setup()

Đây là hàm setup, được Arduino gọi một lần khi khởi động thiết bị. Hàm này sẽ thiết lập các cấu hình cần thiết cho ESP8266 để bắt đầu thực hiện các nhiệm vụ.

2. Serial.begin(9600)

Thiết lập giao tiếp serial với tốc độ truyền dữ liệu là 9600 bps (bit per second). Điều này cho phép ESP8266 giao tiếp với máy tính thông qua cổng serial, giúp bạn in ra các thông báo để kiểm tra quá trình hoạt động của hệ thống.

3. pinMode(LED_PIN, OUTPUT) (và các dòng tương tự cho FAN_PIN, AC_PIN, WARN_PIN)

Thiết lập các chân điều khiển LED, quạt, điều hòa, và cảnh báo làm ngõ ra (OUTPUT).
Cụ thể:

LED_PIN: Chân điều khiển đèn LED.

FAN_PIN: Chân điều khiển quạt.

AC_PIN: Chân điều khiển điều hòa không khí.

WARN_PIN: Chân điều khiển thiết bị cảnh báo.

Tất cả các chân này sẽ được dùng để điều khiển các thiết bị tương ứng bật hoặc tắt.

4. `dht_sensor.begin()`

Khởi tạo cảm biến DHT để bắt đầu thu thập dữ liệu về nhiệt độ và độ ẩm. `dht_sensor` là một đối tượng (object) liên quan đến cảm biến DHT mà bạn đã định nghĩa trước đó.

5. `WiFi.begin(WIFI_SSID, WIFI_PASSWORD)`

Kết nối ESP8266 vào mạng Wi-Fi bằng cách sử dụng SSID (tên mạng) và PASSWORD (mật khẩu) được định nghĩa trước. Quá trình này bắt đầu khởi tạo kết nối Wi-Fi.

6. `while (WiFi.status() != WL_CONNECTED)`

Vòng lặp `while` này sẽ giữ cho chương trình chờ cho đến khi ESP8266 kết nối thành công vào mạng Wi-Fi. Trong mỗi vòng lặp, nó sẽ đợi 500ms và in ra dấu chấm (.) để báo hiệu rằng nó đang cố gắng kết nối.

`WiFi.status()` trả về trạng thái kết nối của Wi-Fi. Nếu trạng thái khác `WL_CONNECTED` (không kết nối), vòng lặp sẽ tiếp tục chờ.

7. `Serial.print("Connected to WiFi network with IP Address: ")`

Khi kết nối Wi-Fi thành công, chương trình sẽ in ra một thông báo cho biết ESP8266 đã kết nối với mạng. Dòng tiếp theo, `Serial.println(WiFi.localIP())`, sẽ in ra địa chỉ IP mà ESP8266 nhận được từ mạng Wi-Fi.

8. `connectToMQTT()`

Cuối cùng, hàm `connectToMQTT()` được gọi để kết nối ESP8266 với MQTT Broker. Hàm này sẽ chịu trách nhiệm thiết lập kết nối MQTT, giúp ESP8266 có thể gửi và nhận dữ liệu qua MQTT.

Hàm `loop()`

```

63 void loop() {
64     mqtt.loop(); // Đảm bảo kết nối MQTT được duy trì
65
66     if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
67         sendToMQTT(); // Gửi dữ liệu lên MQTT mỗi khoảng thời gian nhất định
68         lastPublishTime = millis();
69     }
70
71     // Kiểm tra trạng thái đèn cảnh báo và thực hiện nhấp nháy nếu cần
72     if (isWarningOn) {
73         unsigned long currentMillis = millis();
74         if (currentMillis - previousMillis >= interval) {
75             // Cập nhật thời gian lần nhấp nháy cuối
76             previousMillis = currentMillis;
77
78             // Đảo trạng thái LED cảnh báo
79             ledState = !ledState;
80             digitalWrite(WARN_PIN, ledState);
81         }
82     }
83 }
84

```

1. mqtt.loop();

Dòng này gọi hàm loop() của đối tượng MQTT, đảm bảo rằng kết nối với MQTT Broker luôn được duy trì. Nó xử lý bất kỳ tin nhắn đến nào từ broker và quản lý việc giữ kết nối. Nếu không có dòng này, ESP8266 có thể mất kết nối với broker và không thể nhận hoặc gửi dữ liệu.

2. if (millis() - lastPublishTime > PUBLISH_INTERVAL)

Dòng này kiểm tra xem đã đủ thời gian (theo biến PUBLISH_INTERVAL) kể từ lần cuối cùng dữ liệu được gửi lên MQTT chưa. millis() trả về thời gian (tính bằng milliseconds) từ khi chương trình bắt đầu chạy. Bằng cách lấy hiệu số giữa thời gian hiện tại và thời gian lần cuối gửi dữ liệu (lưu trong lastPublishTime), đoạn mã xác định xem có nên gửi dữ liệu mới không.

3. sendToMQTT();

Nếu thời gian đã đủ để gửi dữ liệu, hàm sendToMQTT() sẽ được gọi. Hàm này thường chứa mã để thu thập dữ liệu từ các cảm biến và gửi nó đến MQTT Broker. Sau khi gửi dữ liệu, thời gian gửi dữ liệu sẽ được cập nhật bằng cách gán lastPublishTime bằng giá trị của millis().

Hàm connect MQTT

```
86 void connectToMQTT() {
87     // Connect to the MQTT broker
88     mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);
89     mqtt.onMessage(messageHandler);
90     Serial.print("ESP8266 - Connecting to MQTT broker");
91     while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
92         Serial.print(".");
93         delay(100);
94     }
95     Serial.println();
96     if (!mqtt.connected()) {
97         Serial.println("ESP8266 - MQTT broker Timeout!");
98         return;
99     }
100    // Subscribe to a topic, the incoming messages are processed by messageHandler() function
101    if (mqtt.subscribe(SUBSCRIBE_TOPIC))
102        Serial.print("ESP8266 - Subscribed to the topic: ");
103    else
104        Serial.print("ESP8266 - Failed to subscribe to the topic: ");
105    Serial.println(SUBSCRIBE_TOPIC);
106    if (mqtt.subscribe("led/req"))
107        Serial.println("ESP8266 - Subscribed to the topic: led/req");
108    else
109        Serial.println("ESP8266 - Failed to subscribe to the topic: led/req");
110    if (mqtt.subscribe("fan/req"))
111        Serial.println("ESP8266 - Subscribed to the topic: fan/req");
112    else
113        Serial.println("ESP8266 - Failed to subscribe to the topic: fan/req");
114    if (mqtt.subscribe("ac/req"))
115        Serial.println("ESP8266 - Subscribed to the topic: ac/req");
116    else
117        Serial.println("ESP8266 - Failed to subscribe to the topic: ac/req");
118    if (mqtt.subscribe("warn/req"))
119        Serial.println("ESP8266 - Subscribed to the topic: ac/req");
120    else
121        Serial.println("ESP8266 - Failed to subscribe to the topic: warn/req");
122    Serial.println("ESP8266 - MQTT broker Connected!");
123 }
124
```

`mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);`

- Khởi tạo kết nối với broker MQTT bằng địa chỉ, cổng và đối tượng mạng (network) đã được xác định trước.

`mqtt.onMessage(messageHandler);`

- Đăng ký hàm `messageHandler` để xử lý các tin nhắn đến từ broker.

Hàm gửi dữ liệu cảm biến

```

void sendToMQTT() {
    StaticJsonDocument<200> message;

    message["light"] = analogRead(LIGHT_SENSOR_PIN); // Đọc giá trị từ cảm biến ánh sáng
    message["humidity"] = dht_sensor.readHumidity(); // Đọc giá trị độ ẩm từ cảm biến DHT
    message["temperature"] = dht_sensor.readTemperature(); // Đọc giá trị nhiệt độ từ cảm biến DHT
    message["dust"] = random(0, 1000); // Tạo giá trị ngẫu nhiên cho độ bụi từ 0 đến 1000

    char messageBuffer[512];
    serializeJson(message, messageBuffer);

    mqtt.publish(PUBLISH_TOPIC, messageBuffer);

    Serial.println("ESP8266 - sent to MQTT:");
    Serial.print("- topic: ");
    Serial.println(PUBLISH_TOPIC);
    Serial.print("- payload:");
    Serial.println(messageBuffer);
}

```

Hàm xử lý tín hiệu từ các topic đã sub

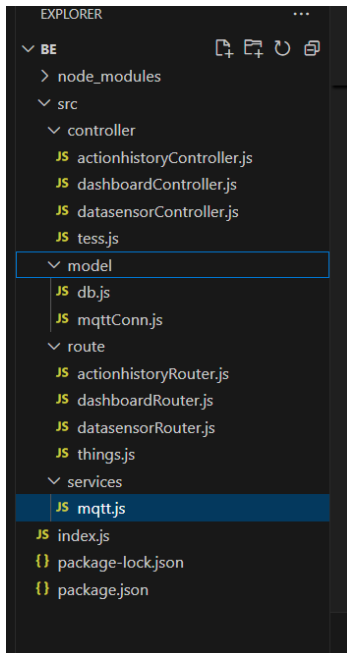
```

145 void messageHandler(String &topic, String &payload) {
146     Serial.println("ESP8266 - received from MQTT:");
147     Serial.println("- topic: " + topic);
148     Serial.print("- payload:");
149     Serial.print(payload);
150     if(topic == "led/req"){
151         handlerLed(payload);
152     }
153     if(topic == "fan/req"){
154         handlerFan(payload);
155     }
156     if(topic == "ac/req"){
157         handlerAc(payload);
158     }
159     if(topic == "warn/req"){
160         handlerWarm(payload);
161     }
162 }
163
164 //ham xu li message control led, fan ,ac khi backend send
165 > void handlerLed(String payload){...
186 }
187 > void handlerFan(String &payload){...
206 }
207 > void handlerAc(String &payload){...
226 }
227
228 | | | | | // Trạng thái của LED
229
230 > void handlerWarm(String &payload) {...
238 }
239

```

4.2 Backend

Cấu trúc thư mục backend:



- Index.js : Chức năng chính của file server.js này là khởi động một server Express để xử lý các yêu cầu HTTP liên quan đến việc điều khiển các thiết bị IoT như quạt, điều hòa, đèn và quản lý dữ liệu cảm biến. Nó kết nối với một cơ sở dữ liệu MongoDB để lưu trữ dữ liệu cảm biến và thiết lập kết nối với MQTT broker để nhận và xử lý thông điệp từ các thiết bị IoT. Ngoài ra, file này cũng thiết lập các tuyến đường (routes) để cung cấp API cho việc điều khiển thiết bị và truy xuất dữ liệu cảm biến.

- Thư mục Model:

- + db.js: file liên kết với database

- + mqttConn: file liên kết với MQTT.

- Thư mục Router: Thư mục này chứa các tệp định nghĩa các tuyến đường (routes) cho ứng dụng, xác định cách xử lý các yêu cầu HTTP tới các endpoint cụ thể. Đồng thời cũng xử lý tạo swagger cho API docs.

- Thư mục Controller: Thư mục này chứa các tệp tin xử lý logic của ứng dụng, bao gồm các hành động liên quan đến việc quản lý yêu cầu từ phía người dùng hoặc các

client khác. Các controller sẽ nhận và xử lý các yêu cầu HTTP từ các route, sau đó gọi đến các service hoặc model tương ứng để thực hiện các tác vụ cần thiết.

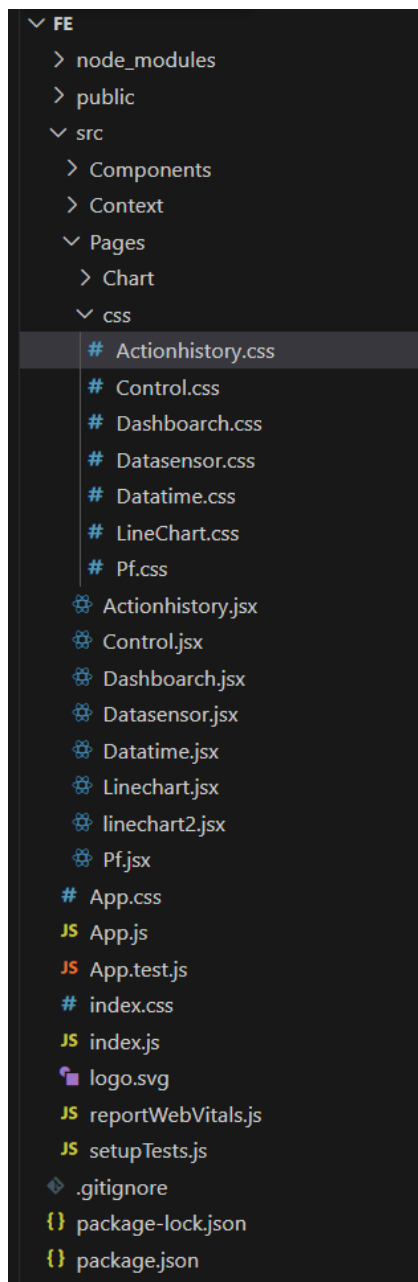
- Thư mục services: nơi khởi tạo Websocket, nhận dữ liệu cảm biến từ phần cứng và gửi về frontend.

4.2.2. Luồng hoạt động:

- Khi hệ thống nhận một yêu cầu HTTP (ví dụ: bật quạt), yêu cầu này sẽ được xử lý bởi một controller tương ứng trong thư mục Controllers.
- Controller sẽ gọi đến các model trong Models nếu cần truy xuất hoặc cập nhật dữ liệu trong MongoDB.
- Nếu yêu cầu liên quan đến việc giao tiếp với các thiết bị IoT, controller sẽ tương tác với các module trong Mqtt để gửi thông điệp qua MQTT hoặc nhận phản hồi từ các thiết bị.

4.3 Frontend

Cấu trúc thư mục



File App.js: App.js là file chính của ứng dụng React, là nơi tất cả các component khác được kết hợp lại để tạo ra ứng dụng hoàn chỉnh.

```

2  import './App.css';
3  import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
4  import Navbar from './Components/Navbar/Navbar';
5  import Pf from './Pages/Pf';
6  import Dashboarch from './Pages/Dashboarch';
7  import Datasensor from './Pages/Datasensor';
8  import Actionhistory from './Pages/Actionhistory';
9
10 function App() {
11   return (
12     <div className="App">
13       <Router>
14         <Navbar />
15         <Routes>
16           <Route path="/" element={<Dashboarch />}></Route>
17           <Route path="/datasensor" element={<Datasensor />}></Route>
18
19           <Route path="/actionhistory" element={<Actionhistory />}></Route>
20           <Route path="/profile" element={<Pf />}></Route>
21         </Routes>
22       </Router>
23     </div>
24   );
25 }
26
27 export default App;
28

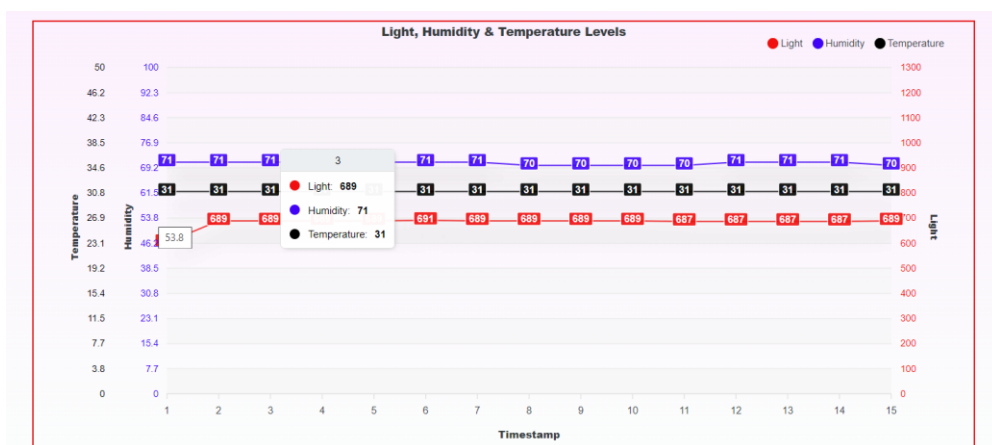
```

Dashboarch, Datasensor, Actionhistory, Pf lần lượt ứng với trang chính, trang dữ liệu cảm biến, trang lịch sử hành động và cuối cùng là trang cá nhân.

- Trang Dashboard



Biểu đồ hiển thị: hệ thống cũng cung cấp đồ thị thời gian thực, phản ánh sự biến đổi của các giá trị này theo từng thời điểm, giúp người dùng dễ dàng quan sát và phân tích dữ liệu.



Trang datasensor: Trang này chứa thông tin về các thông số như nhiệt độ, độ ẩm, và ánh sáng đã được lưu trữ từ trước đến nay.

Người dùng có thể xem lại lịch sử các chỉ số này, giúp theo dõi sự thay đổi và đánh giá tình trạng môi trường qua thời gian.

ID	Temperature	Light	Dust	Humidity	Time
1	26	885	624	40	2024-10-09 07:39:20
2	26	885	399	40	2024-10-09 07:39:25
3	26	885	38	40	2024-10-09 07:39:30
4	26	885	953	40	2024-10-09 07:39:35
5	26	885	434	40	2024-10-09 07:41:54
6	26	885	509	40	2024-10-09 07:41:59
7	26	885	98	40	2024-10-09 07:42:04
8	26	885	900	40	2024-10-09 07:42:04
9	26	885	550	40	2024-10-09 07:42:09
10	26	885	76	41	2024-10-09 07:44:10

Trang Actionhistory: Trang này lưu trữ và hiển thị thông tin về lịch sử các hành động bật/tắt thiết bị từ trước đến nay. Người dùng có thể kiểm tra các thao tác điều khiển thiết bị đã thực hiện, đảm bảo rằng mọi thay đổi trạng thái của các thiết bị đều được ghi lại và có thể kiểm tra lại khi cần thiết.



Dashboard Datasensor Action history

 Profile

Page Size: 10

ID	Device	On-Time	Off-Time
1	fan	on	2024-09-25 11:39:54
2	fan	off	2024-09-25 11:39:56
3	led	on	2024-09-25 11:39:59
4	led	off	2024-09-25 11:40:00
5	ac	on	2024-09-25 11:40:29
6	ac	off	2024-09-25 11:40:30
7	led	on	2024-09-25 11:40:39
8	led	off	2024-09-25 11:40:40
9	led	on	2024-09-25 11:43:40
10	led	off	2024-09-25 11:43:42

Trang 1 / 43

Go to Page: